
OPENCV Y ZED2 DESDE PYTHON:

INTRODUCCIÓN A LA CAPTURA DE IMÁGENES

INTRODUCCIÓN

OpenCV (Open Source Computer Vision) es una librería de programación de visión por computador escrita en el lenguaje C++. Se caracteriza por su gran eficiencia y es ampliamente utilizada debido a su constante desarrollo realizado por la comunidad Open Source. Se trata de una librería multiplataforma y debido a su gran popularidad tiene interfaces para su utilización en otros lenguajes de programación como Java, Python, C# o Matlab entre otros. Fue originalmente creada en 1999 por Intel y actualmente es mantenida por la empresa Itseez Inc.

El SDK de ZED (de stereolabs) es un sistema de desarrollo de visión artificial avanzado, escalable y personalizable para la industria, utilizado por más de 100.000 usuarios en todo el mundo. Ofrece múltiples funciones que le permiten crear aplicaciones de visión de forma sencilla.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es diseñar una interfaz genérica de captura de imágenes, que sirva para los distintos tipos de cámara. Se parte de un material base donde se presenta cómo se puede adquirir y gestionar imágenes desde Python utilizando OpenCV 4.5 y ZED2.

Este objetivo se descompone en los siguientes objetivos concretos:

1. Introducción a OpenCV desde Python.
2. Introducción a ZED2 desde Python.
3. Adquisición de imágenes.
4. Gestión de imágenes.

MATERIALES

Descargue los siguientes materiales a una carpeta de trabajo local (en adelante, "[working_folder_path](#)") en el ordenador donde se vaya a ejecutar el cuaderno.

- PoliformaT -> Recursos/2-Software-&-cuadernos/20-Jupyter lab Tutorial – Digital image
- PoliformaT -> Recursos/2-Software-&-cuadernos/21-Jupyter lab Tutorial – Image acquisition
- PoliformaT -> Recursos/2-Software-&-cuadernos/22-ZED2 sample – Image acquisition

Importante >> El último cuaderno solo se podrá ejecutar en uno de los servidores (con GPU). No olvide hacer una copia de seguridad del código implementado durante la práctica (por ejemplo, en su carpeta W), ya que los ordenadores del laboratorio se limpiarán periódicamente.

1.1- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

```
(base) C:\windows\system32> conda activate open3d_env  
(open3d_env) C:\Users\username> jupyter lab --notebook-dir "working_folder_path"
```

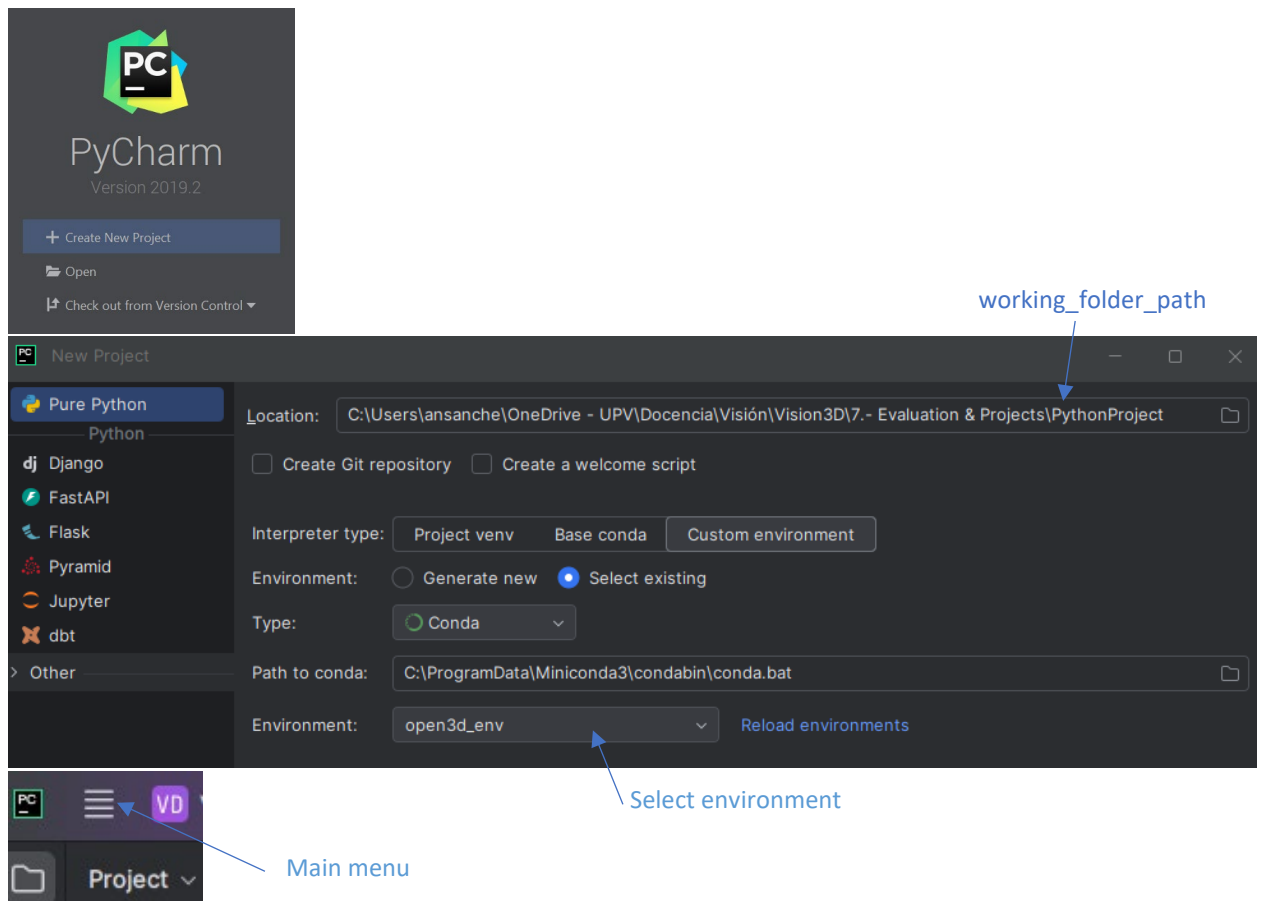
Después de arrancar Jupyter lab, escribe en una célula de código el comando: "import cv2"

Mediante este módulo, Python encontrará todos los métodos de Python que sirven de interfaz entre la librería OpenCV de C++ y Python.

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

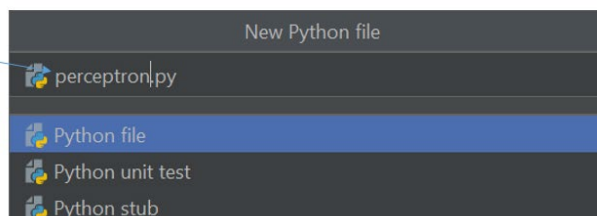
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



2.- CAPTURA DE IMÁGENES CON OPENCV

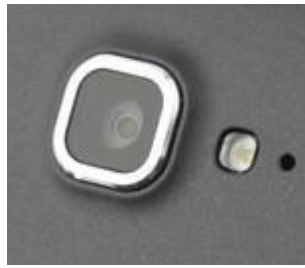


Fig. 1.- Sistema de visión personal (webcam o móvil).

En el cuaderno "20_Digital_image.ipynb", se muestra el código básico de Python que permite leer, escribir y visualizar imágenes, utilizando la funcionalidad de Opencv.

Por otro lado, en el cuaderno "21_Image_acquisition.ipynb", se muestra el código básico de Python que permite capturar imágenes de la webcam o de una cámara IP.

Si desea utilizar la cámara de su teléfono móvil, como una cámara IP, deberá instalar y ejecutar la correspondiente App.

IPWEBCAM *(Android app)*

PoliformaT -> smartwebcam.apk

<https://tinyurl.com/72wnw3f>

<https://play.google.com/store/apps/details?id=com.dev47apps.obs-droidcam&hl=es> 419



DroidCam *(Iphone app)*

<https://apps.apple.com/us/app/droidcam-webcam-obs-camera/id1510258102>



3.- CAPTURA DE IMÁGENES CON ZED2

ZED SDK ofrece diversos ejemplos de código para facilitar su aprendizaje. Todos estos ejemplos se pueden encontrar en el siguiente enlace [Stereolabs GitHub page](#).



Stereolabs® ZED™ (Stereo)

Fig. 2.- Cámara ZED2.

En el cuaderno "22_Image_acquisition_ZED2.ipynb", se muestra el código básico de Python que permite capturar imágenes de la cámara ZED2, utilizando la funcionalidad de OpenCV.

Importante >> Para poder utilizar una cámara ZED2 del laboratorio será necesario conectarse y desconectarse por turnos a un servidor según se explica en el documento "Indicaciones de uso de servidores de cámara para V3D.pdf". En los servidores se dispone de la carpeta compartida V3D_DATASETS donde todos los equipos de trabajo compartiréis las imágenes adquiridas en directorios diferentes.

EJERCICIO 3.1:

Desarrolle un método de la Clase Camera en Python que haga una adquisición continua de una cámara hasta que el usuario pulse la tecla 'Esc' or 'q'. Este código guardará, en un directorio especificado por el usuario, la secuencia de imágenes capturadas por la cámara a una velocidad de fotogramas (fps) determinada. Cada imagen se guardará como un archivo BMP con nombres de archivo numerados secuencialmente (rellenados con ceros). Por ejemplo 000000.bmp, 000001.bmp, ... Además, a la vez que se guardan las imágenes, estas se mostrarán en una ventana OpenCV hasta que se presione la tecla 'Esc' or 'q'.

A nivel de gestión de errores, el código creará la estructura de directorio requerida si esta no existe.

```
class Camera():
    """
    Represents a camera object for video capture.

    This class is responsible for managing a camera, including initialization,
    and image reading. It provides methods to handle video streams. The class also supports
    saving sequences to disk and displaying live feed from the camera.

    :ivar name: Name of the camera.
    :type name: str
    :ivar video: Video capture object for accessing the camera or video file.
    :type video: cv2.VideoCapture
    :ivar runtime: Placeholder for runtime information, currently unused.
    :type runtime: any
    :ivar shape: Dimensions of the image expressed as (width, height).
    :type shape: tuple[int, int]
    """

    def __init__(self, name, camera):
        self.name = 'camera_' + name

    def save_sequence(self, sequence_name, fps=1):
        """
        Saves frames captured from the camera at a fixed frame rate as sequential
        image files in the specified directory. Each frame is saved as a BMP file with
        sequentially numbered filenames (zero-padded). The function also displays the
        captured frames in a OpenCV window until either the 'Esc' key or the 'q' key is pressed.

        The function creates the required directory structure if it does not exist.

        :param sequence_name: The name of the directory where frames will be saved.
        :type sequence_name: str
        :param fps: The frame rate at which frames are extracted and saved. Defaults to 1.
        :type fps: int
        :return: None
        """
```

Los comandos de Python que se pueden utilizar para gestionar paths:

```
>>> import os
if not os.path.exists(path):
    os.makedirs(path)
```

Los comandos de Python y Opencv para leer de teclado:

```
>>> import cv2
>>> key = cv2.waitKey(1) -> 'ESC' =27, 'q'=113
```

Los comandos de Python que se pueden utilizar para gestionar tiempos de espera son:

```
>>> import time
>>> start = time.time()
>>> time.sleep(delay) # delay in secs
>>> stop = time.time()
>>> delay_est = stop - start
```

En el caso de que no dispongas de una cámara física para probar la adquisición, puedes realizar el ejercicio 3.2 con las secuencias de imágenes guardadas en la carpeta "camera_zed1_LEFT" y "camera_zed1_RIGHT".

EJERCICIO 3.2:

Desarrolle un método de la clase Camera en Python para leer secuencias de imágenes de fichero. Si se le pasa una ruta a un fichero mostrará la imagen. Si se le pasa un directorio leerá todas las imágenes del directorio y las irá reproduciendo a una velocidad de fotogramas (fps) determinada. Para distinguir si es una imagen o un conjunto de imágenes se comprobará si tiene extensión o no el argumento de entrada de la función.

Los comandos de Python que se pueden utilizar para obtener los ficheros que se encuentran en un directorio son:

```
>>> import os
>>> contenido = os.listdir('Path_to_folder')
```

o también:

```
>>> import glob
>>> images = glob.glob('Path_to_folder'+ "/*.bmp")
```

4.- PREPROCESO DE IMÁGENES CON FUNCIONALIDAD OPENCV: ESPACIOS DE COLOR

EJERCICIO 4.1:

Desarrolle un método de la clase Camera en Python que lea imágenes de una secuencia y muestre secuencialmente los diferentes canales (uno por ventana con su correspondiente título). El cambio entre los espacios de color se realizará cuando el usuario apriete la tecla 'c'.

Para cambiar de espacios de color se utilizará la función " result = cv2.cvtColor(imagen,código)" donde los códigos posibles son: [cv2.COLOR_BGR2HSV, cv2.COLOR_BGR2XYZ, cv2.COLOR_BGR2YCrCb, cv2.COLOR_BGR2YUV, cv2.COLOR_BGR2HLS, cv2.COLOR_BGR2Lab, cv2.COLOR_BGR2Luv].

EJERCICIO 4.2:

Desarrolle un método de la clase Camera en Python que lea imágenes de una secuencia y muestre secuencialmente el histograma de cada imagen.